

Artificial Intelligence and Data Engineering for Orbital and Space Observation Systems

Laboratory 11: Semantic Segmentation for Earth Observation

Leszek Siwik
Faculty of Space Technologies
AGH University of Krakow

1 Introduction

In previous laboratories, you worked with satellite image classification.

A classification model answers:

```
What is visible in this image?
```

For example:

```
Input image -> river  
Input image -> forest  
Input image -> residential
```

In Laboratory 10, you also investigated explainability.

You used Grad-CAM and occlusion sensitivity to ask:

```
Why did the model make this prediction?  
Where did the model focus its attention?
```

In this laboratory, you will move from image-level prediction to pixel-level prediction.

This task is called:

```
semantic segmentation
```

Semantic segmentation assigns a class label to every pixel in the image.

Instead of producing one label for the whole image, the model produces a mask.

```
Classification:  
image -> one label  
  
Semantic segmentation:  
image -> label for each pixel
```

For Earth Observation systems, semantic segmentation is one of the most important AI tasks. It can be used for:

- land-cover mapping,
- water body detection,

- urban area extraction,
- vegetation monitoring,
- disaster assessment,
- change detection,
- mission planning and situational awareness.

2 Goal of This Laboratory

The goal of this laboratory is to build a first complete semantic segmentation pipeline.

You will create a small synthetic Earth Observation dataset.

Each sample will contain:

- an artificial satellite-like image,
- a corresponding segmentation mask.

The image will contain simplified regions representing:

- background terrain,
- vegetation,
- water,
- urban structures.

You will train a small U-Net-like neural network to predict segmentation masks.

The pipeline will be:

```
generate synthetic EO dataset
-> create Dataset class
-> create DataLoaders
-> build segmentation model
-> train model
-> evaluate pixel-level accuracy
-> visualize predictions
-> analyze segmentation errors
```

3 Why Synthetic Data?

Real semantic segmentation datasets for Earth Observation may be large and complex.

They often require:

- downloading large files,
- working with geospatial formats,
- handling multispectral bands,
- processing large masks,
- using specialized GIS tools.

In this laboratory, we use a synthetic dataset to focus on the core AI concepts:

- image-mask pairs,
- pixel-level labels,
- segmentation model output,
- loss function for segmentation,
- visualization of predicted masks.

Synthetic data is also useful in space systems engineering.

It can support:

- simulation-based development,
- early algorithm prototyping,
- testing pipelines before real data is available,
- generating rare or controlled scenarios.

4 Environment Setup

Install required libraries:

```
python3 -m pip install torch torchvision pillow numpy matplotlib scikit-learn
```

All commands should be executed from the root directory of your project.

5 Project Structure

Create the following structure:

```
data/  
  segmentation/  
    images/  
    masks/  
  
models/  
  
reports/  
  segmentation_examples/  
  
src/  
  segmentation/  
    generate_synthetic_dataset.py  
    segmentation_dataset.py  
    unet_model.py  
    train_segmentation.py  
    predict_segmentation.py
```

Create folders:

```
mkdir -p data/segmentation/images data/segmentation/masks  
mkdir -p models reports/segmentation_examples  
mkdir -p src/segmentation
```

6 Task 1: Generate Synthetic Segmentation Dataset

Create:

```
src/segmentation/generate_synthetic_dataset.py
```

Add imports:

```
from pathlib import Path
import random

import numpy as np
from PIL import Image, ImageDraw
```

Add configuration:

```
IMAGE_DIR = Path("data/segmentation/images")
MASK_DIR = Path("data/segmentation/masks")

IMAGE_SIZE = 128
NUM_IMAGES = 200
RANDOM_SEED = 42
```

This configuration defines where generated files will be stored, how large each image will be and how many image-mask pairs will be created. The same class identifiers will later be used by the Dataset, the model output and the loss function.

In this dataset, mask classes will be:

```
0 -> background
1 -> vegetation
2 -> water
3 -> urban
```

Add:

```
CLASS_COLORS = {
    0: (80, 80, 80),      # background
    1: (40, 140, 40),     # vegetation
    2: (40, 80, 180),     # water
    3: (180, 180, 180)    # urban
}
```

7 Task 2: Create One Synthetic Scene

Add:

```
def create_scene(index):
    image = Image.new(
        "RGB",
        (IMAGE_SIZE, IMAGE_SIZE),
        CLASS_COLORS[0]
    )

    mask = Image.new(
        "L",
        (IMAGE_SIZE, IMAGE_SIZE),
```

```

0
)

image_draw = ImageDraw.Draw(image)
mask_draw = ImageDraw.Draw(mask)

# vegetation region
x1 = random.randint(0, 40)
y1 = random.randint(0, 40)
x2 = random.randint(70, 127)
y2 = random.randint(70, 127)

image_draw.ellipse(
    [x1, y1, x2, y2],
    fill=CLASS_COLORS[1]
)

mask_draw.ellipse(
    [x1, y1, x2, y2],
    fill=1
)

# water region
river_x = random.randint(40, 80)

image_draw.rectangle(
    [river_x, 0, river_x + 15, IMAGE_SIZE],
    fill=CLASS_COLORS[2]
)

mask_draw.rectangle(
    [river_x, 0, river_x + 15, IMAGE_SIZE],
    fill=2
)

# urban blocks
for _ in range(5):
    ux = random.randint(0, IMAGE_SIZE - 20)
    uy = random.randint(0, IMAGE_SIZE - 20)

    image_draw.rectangle(
        [ux, uy, ux + 12, uy + 12],
        fill=CLASS_COLORS[3]
    )

    mask_draw.rectangle(
        [ux, uy, ux + 12, uy + 12],
        fill=3
    )

# add slight RGB noise
image_array = np.array(image).astype(np.int16)

noise = np.random.randint(
    -15,
    16,
    image_array.shape
)

```

```

image_array = np.clip(
    image_array + noise,
    0,
    255
).astype(np.uint8)

image = Image.fromarray(image_array)

image.save(
    IMAGE_DIR / f"scene_{index:04d}.png"
)

mask.save(
    MASK_DIR / f"scene_{index:04d}.png"
)

```

8 Task 3: Generate Full Dataset

Add:

```

def main():
    random.seed(RANDOM_SEED)
    np.random.seed(RANDOM_SEED)

    IMAGE_DIR.mkdir(
        parents=True,
        exist_ok=True
    )

    MASK_DIR.mkdir(
        parents=True,
        exist_ok=True
    )

    for index in range(NUM_IMAGES):
        create_scene(index)

    print("=== Synthetic Segmentation Dataset ===")
    print(f"Generated images: {NUM_IMAGES}")
    print(f"Image folder: {IMAGE_DIR}")
    print(f"Mask folder: {MASK_DIR}")

if __name__ == "__main__":
    main()

```

Run:

```
python3 -m src.segmentation.generate_synthetic_dataset
```

Expected output:

```

=== Synthetic Segmentation Dataset ===
Generated images: 200
Image folder: data/segmentation/images
Mask folder: data/segmentation/masks

```

9 What Was Actually Generated?

Before continuing, it is important to understand what the dataset generation step produced. The script generated:

- 200 synthetic satellite-like images,
- 200 corresponding segmentation masks.

Each image-mask pair represents one training example.

For example:

```
images/scene_0000.png  
masks/scene_0000.png
```

Although both files have the same name, they contain different information.

The Input Image

The image represents a simplified Earth Observation scene.

Several artificial land-cover regions are generated:

- gray background terrain,
- green vegetation area,
- blue river,
- white urban structures.

Random noise is added to make the task slightly more realistic.

The generated image may look similar to:

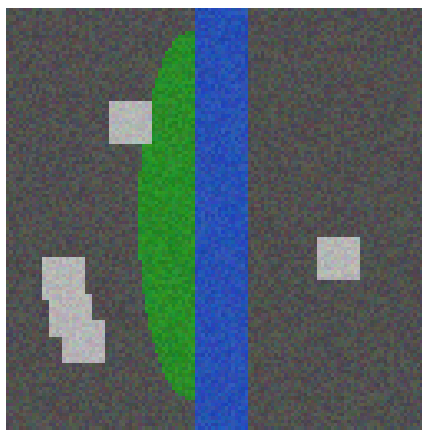


Figure 1: Sample input image

This image will be used as the input to the neural network.

The Segmentation Mask

Unlike a regular RGB image, a segmentation mask stores class identifiers rather than colors.

For example:

```
0 -> background
1 -> vegetation
2 -> water
3 -> urban
```

A fragment of a mask might look like:

```
0 0 0 0 2 2 2
0 1 1 1 2 2 2
0 1 1 1 0 0 3
```

Every pixel already has a known class label.

The mask therefore represents the ground truth.

Figure 2 shows one example generated by the synthetic dataset generator.

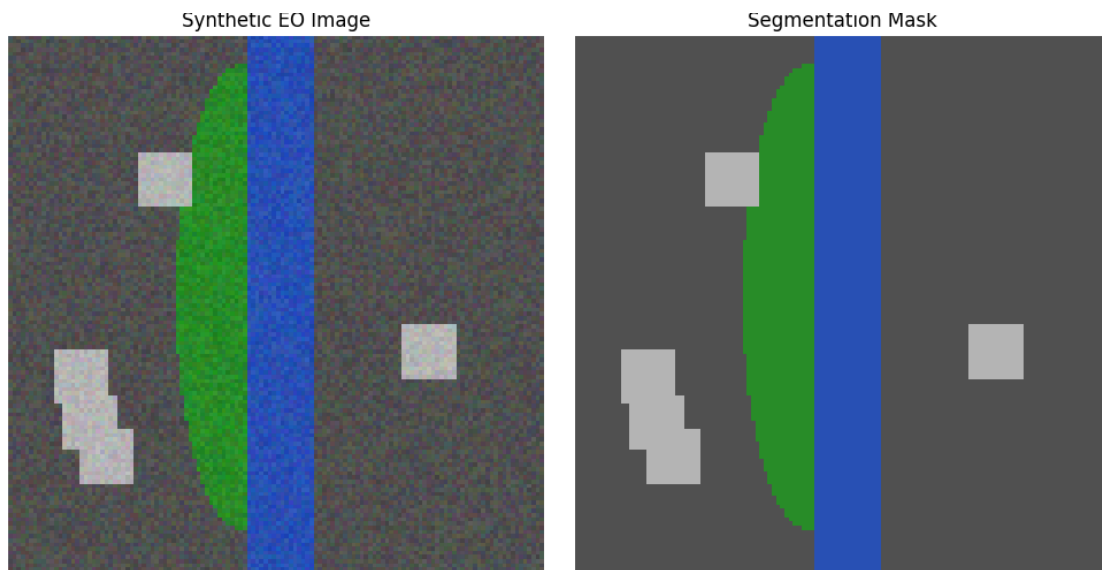


Figure 2: Example image-mask pair generated for semantic segmentation training.

The left image represents a simplified Earth Observation scene.

Several land-cover categories were generated:

- gray terrain (background),
- green vegetation,
- blue river,
- white urban structures.

Random noise was added to the image to simulate measurement uncertainty and sensor variability.

The right image is the corresponding segmentation mask.

Unlike the input image, the mask contains the correct class label for every pixel.

The color coding is used only for visualization:

- gray = background,

- green = vegetation,
- blue = water,
- white = urban area.

Why Do We Need Masks?

Unlike image classification, semantic segmentation requires pixel-level supervision.
For classification, training data looks like:

```
image -> river
image -> forest
image -> residential
```

For segmentation, training data looks like:

```
image
|
mask
|
pixel labels
```

The neural network learns by comparing:

- predicted segmentation mask,
- ground-truth segmentation mask.

Training attempts to minimize the difference between them.

How Does the Network Learn?

Suppose the ground-truth mask contains:

```
water
water
water
```

but the model predicts:

```
background
water
background
```

The loss function penalizes these mistakes.

After many training iterations, the model gradually learns:

- how vegetation looks,
- how rivers look,
- how urban structures look,
- where object boundaries occur.

10 Task 4: Visualizing Generated Masks

Before building a segmentation dataset and training a neural network, it is useful to inspect several generated samples.

When opening a generated mask using a standard image viewer, you may notice that it appears almost completely black.

This is expected.

The mask does not store colors.

Instead, it stores class identifiers:

```
0 -> background
1 -> vegetation
2 -> water
3 -> urban
```

These values are much smaller than typical RGB intensities:

```
0 ... 255
```

As a result:

- class 0 appears black,
- class 1 appears almost black,
- class 2 appears almost black,
- class 3 appears almost black.

The mask is intended for machine learning rather than human interpretation.

To better understand the generated dataset, we will create a visualization utility that converts class identifiers into meaningful colors.

Create Visualization Script

Create:

```
src/segmentation/visualize_mask.py
```

Add:

```
from pathlib import Path

import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

IMAGE_PATH = Path(
    "data/segmentation/images/scene_0000.png"
)

MASK_PATH = Path(
    "data/segmentation/masks/scene_0000.png"
)

OUTPUT_PATH = Path(
```

```

    "reports/segmentation_examples/"
    "dataset_example.png"
)

PALETTE = {
    0: (80, 80, 80),      # background
    1: (40, 140, 40),    # vegetation
    2: (40, 80, 180),    # water
    3: (180, 180, 180)   # urban
}

def mask_to_rgb(mask):

    height, width = mask.shape

    rgb = np.zeros(
        (height, width, 3),
        dtype=np.uint8
    )

    for class_id, color in PALETTE.items():

        rgb[
            mask == class_id
        ] = color

    return rgb

def main():

    if not IMAGE_PATH.exists():

        print(
            f"Error: image not found: {IMAGE_PATH}"
        )

        print(
            "Run generate_synthetic_dataset.py first."
        )

        raise SystemExit(1)

    if not MASK_PATH.exists():

        print(
            f"Error: mask not found: {MASK_PATH}"
        )

        print(
            "Run generate_synthetic_dataset.py first."
        )

        raise SystemExit(1)

    image = Image.open(
        IMAGE_PATH

```

```

).convert("RGB")

mask = np.array(
    Image.open(MASK_PATH),
    dtype=np.uint8
)

mask_rgb = mask_to_rgb(
    mask
)

plt.figure(
    figsize=(10,5)
)

plt.subplot(1,2,1)

plt.imshow(image)

plt.title(
    "Synthetic EO Image"
)

plt.axis("off")

plt.subplot(1,2,2)

plt.imshow(mask_rgb)

plt.title(
    "Colorized Segmentation Mask"
)

plt.axis("off")

plt.tight_layout()

OUTPUT_PATH.parent.mkdir(
    parents=True,
    exist_ok=True
)

plt.savefig(
    OUTPUT_PATH,
    dpi=300,
    bbox_inches="tight"
)

plt.show()

print(
    f"Saved: {OUTPUT_PATH}"
)

if __name__ == "__main__":
    main()

```

Run:

```
python3 -m src.segmentation.visualize_mask
```

Expected output:

```
Saved:
reports/segmentation_examples/dataset_example.png
```

Inspecting Generated Samples

The generated visualization should contain:

- the original synthetic Earth Observation image,
- the corresponding colorized segmentation mask.

The left image represents the model input.

The right image represents the correct answer that the model will learn to predict.

Verify that:

- the river appears in the same location in both images,
- vegetation regions correspond between image and mask,
- urban structures appear in both images,
- every visible structure has a corresponding semantic label.

Observe that the image contains visual information such as:

- colors,
- textures,
- noise,
- object shapes.

The mask contains only semantic information:

- what each pixel represents,
- where class boundaries occur.

What Will Happen Next?

The visualization confirms that every image has a corresponding segmentation mask.

In the next steps:

- images will be converted into tensors,
- masks will be converted into label maps,
- a U-Net model will be trained,
- the model will learn to predict masks from images.

The image-mask pairs inspected in this task will become the training examples used by the segmentation model.

11 Task 5: Create Segmentation Dataset Class

Create:

```
src/segmentation/segmentation_dataset.py
```

Add:

```
from pathlib import Path

import torch
from torch.utils.data import Dataset
from PIL import Image
import numpy as np
from torchvision import transforms
```

Add:

```
class SyntheticSegmentationDataset(Dataset):

    def __init__(
        self,
        image_dir,
        mask_dir,
        transform=None
    ):
        self.image_dir = Path(image_dir)
        self.mask_dir = Path(mask_dir)
        self.transform = transform

        self.image_paths = sorted(
            self.image_dir.glob("*.png")
        )

        if len(self.image_paths) == 0:
            print("Error: no images found.")
            print("Run generate_synthetic_dataset.py first.")
            raise SystemExit(1)

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, index):
        image_path = self.image_paths[index]

        mask_path = (
            self.mask_dir
            / image_path.name
        )

        image = Image.open(
            image_path
        ).convert("RGB")

        mask = Image.open(
            mask_path
        )
```

```

        if self.transform is not None:
            image = self.transform(image)
        else:
            image = transforms.ToTensor()(image)

        mask_array = np.array(
            mask,
            dtype=np.int64
        )

        mask_tensor = torch.from_numpy(
            mask_array
        )

        return image, mask_tensor

```

Add inspection code:

```

def main():
    dataset = SyntheticSegmentationDataset(
        image_dir="data/segmentation/images",
        mask_dir="data/segmentation/masks"
    )

    image, mask = dataset[0]

    print("=== Segmentation Dataset Inspection ===")
    print(f"Number of samples: {len(dataset)}")
    print(f"Image shape: {image.shape}")
    print(f"Mask shape: {mask.shape}")
    print(f"Mask classes: {torch.unique(mask)}")

if __name__ == "__main__":
    main()

```

Run:

```
python3 -m src.segmentation.segmentation_dataset
```

Expected output:

```

=== Segmentation Dataset Inspection ===
Number of samples: 200
Image shape: torch.Size([3, 128, 128])
Mask shape: torch.Size([128, 128])
Mask classes: tensor([0, 1, 2, 3])

```

What Just Happened?

At this stage, we are no longer working directly with image files.

The purpose of the Dataset class is to convert files stored on disk into tensors that can be processed by a neural network.

The workflow is:

```
PNG files
```

```
|  
Dataset  
|  
PyTorch tensors  
|  
DataLoader  
|  
Neural Network
```

The Dataset class acts as a bridge between the filesystem and the machine learning model.

Why Did We Generate Images Earlier?

In previous tasks, we generated:

```
data/segmentation/images  
data/segmentation/masks
```

These files represent the training dataset.

However, neural networks cannot directly operate on PNG files.

Before training begins, the images must be loaded into memory and converted into numerical tensors.

This is exactly the purpose of:

```
SyntheticSegmentationDataset
```

The Dataset class does not create new images.

Instead, it loads the images generated earlier and converts them into a format suitable for machine learning.

Understanding the Output

The Dataset inspection produced:

```
Number of samples: 200
```

This means:

- 200 image-mask pairs were found,
- the dataset generation step was successful,
- every training example is available for loading.

Image Shape

The inspection also displayed:

```
Image shape:  
torch.Size([3, 128, 128])
```

This tensor represents:

```
[channels, height, width]
```

Therefore:

- 3 channels (RGB),
- height = 128 pixels,
- width = 128 pixels.

The image has therefore been converted from:

```
scene_0000.png
```

into a numerical tensor:

```
3  128  128
```

which can be processed by a neural network.

Mask Shape

The inspection also displayed:

```
Mask shape:
torch.Size([128, 128])
```

Unlike RGB images, segmentation masks do not contain color channels.

Each pixel stores only a class identifier.

For example:

```
0 0 0 2 2 2
0 1 1 2 2 2
0 1 1 0 0 3
```

Therefore, only height and width are needed.

Mask Classes

The final output was:

```
Mask classes:
tensor([0, 1, 2, 3])
```

This confirms that all expected classes are present in the dataset:

- 0 = background,
- 1 = vegetation,
- 2 = water,
- 3 = urban.

This is an important sanity check.

If a class were missing, the model would never learn to recognize it.

Why Is This Inspection Important?

Before training any machine learning model, engineers typically verify:

- dataset size,

- tensor dimensions,
- class labels,
- annotation consistency.

Many training failures can be traced back to problems detected at this stage.

Performing dataset inspection before training is therefore considered good engineering practice.

12 Task 6: Create a Small U-Net Model

U-Net is a common neural network architecture for image segmentation.

It has two main parts:

- encoder: reduces spatial size and extracts features,
- decoder: reconstructs pixel-level output.

The intuition is:

```
input image
|
encoder
|
compressed features
|
decoder
|
segmentation mask
```

One of the most important characteristics of U-Net is the use of skip connections.

During downsampling, the encoder learns increasingly abstract representations of the image.

However, some spatial details may be lost.

Skip connections transfer feature maps directly from the encoder to the decoder:

```
encoder features
    |
    +-----+
              |
decoder <-----+
```

This helps the decoder reconstruct precise object boundaries and preserve small structures.

Skip connections are one of the main reasons why U-Net performs well on semantic segmentation tasks.

Create:

```
src/segmentation/unet_model.py
```

Add:

```
import torch
from torch import nn
```

Add convolution block:

```

class ConvBlock(nn.Module):

    def __init__(
        self,
        in_channels,
        out_channels
    ):
        super().__init__()

        self.block = nn.Sequential(
            nn.Conv2d(
                in_channels,
                out_channels,
                kernel_size=3,
                padding=1
            ),
            nn.ReLU(),
            nn.Conv2d(
                out_channels,
                out_channels,
                kernel_size=3,
                padding=1
            ),
            nn.ReLU()
        )

    def forward(self, x):
        return self.block(x)

```

Add model:

```

class SmallUNet(nn.Module):

    def __init__(
        self,
        num_classes
    ):
        super().__init__()

        self.encoder1 = ConvBlock(
            3,
            16
        )

        self.pool1 = nn.MaxPool2d(2)

        self.encoder2 = ConvBlock(
            16,
            32
        )

        self.pool2 = nn.MaxPool2d(2)

        self.bottleneck = ConvBlock(
            32,
            64
        )

```

```

        self.up2 = nn.ConvTranspose2d(
            64,
            32,
            kernel_size=2,
            stride=2
        )

        self.decoder2 = ConvBlock(
            64,
            32
        )

        self.up1 = nn.ConvTranspose2d(
            32,
            16,
            kernel_size=2,
            stride=2
        )

        self.decoder1 = ConvBlock(
            32,
            16
        )

        self.output = nn.Conv2d(
            16,
            num_classes,
            kernel_size=1
        )

    def forward(self, x):

        enc1 = self.encoder1(x)

        x = self.pool1(enc1)

        enc2 = self.encoder2(x)

        x = self.pool2(enc2)

        x = self.bottleneck(x)

        x = self.up2(x)

        x = torch.cat(
            [x, enc2],
            dim=1
        )

        x = self.decoder2(x)

        x = self.up1(x)

        x = torch.cat(
            [x, enc1],
            dim=1
        )

```

```
x = self.decoder1(x)

x = self.output(x)

return x
```

Add inspection:

```
def main():
    model = SmallUNet(
        num_classes=4
    )

    sample = torch.randn(
        1,
        3,
        128,
        128
    )

    output = model(sample)

    print("=== U-Net Inspection ===")
    print(f"Input shape: {sample.shape}")
    print(f"Output shape: {output.shape}")

if __name__ == "__main__":
    main()
```

Run:

```
python3 -m src.segmentation.unet_model
```

Expected output:

```
=== U-Net Inspection ===
Input shape: torch.Size([1, 3, 128, 128])
Output shape: torch.Size([1, 4, 128, 128])
```

What Just Happened?

This script does not train the model yet.

It only checks whether the U-Net architecture can process an input image and produce an output with the correct shape.

The model received a dummy input tensor:

```
torch.Size([1, 3, 128, 128])
```

This means:

- 1 image in the batch,
- 3 input channels: red, green and blue,

- image height: 128 pixels,
- image width: 128 pixels.

The model returned:

```
torch.Size([1, 4, 128, 128])
```

This means:

- 1 image in the batch,
- 4 output channels,
- 128x128 spatial resolution.

Why Are There 4 Output Channels?

The output has 4 channels because the segmentation problem has 4 classes:

```
0 -> background
1 -> vegetation
2 -> water
3 -> urban
```

For every pixel, the model produces 4 numerical scores.

For example, for one pixel, the model may output:

```
background score: 1.2
vegetation score: 0.4
water score:      3.8
urban score:      0.1
```

The highest score corresponds to the predicted class.

In this example, the pixel would be classified as:

```
water
```

because the water score is the highest.

From Scores to a Segmentation Mask

The raw model output is not yet a segmentation mask.

It is a tensor of class scores.

To convert it into a mask, we select the class with the highest score for every pixel:

```
prediction = torch.argmax(output, dim=1)
```

This transforms:

```
[1, 4, 128, 128]
```

into:

```
[1, 128, 128]
```

The result is a predicted mask where each pixel contains one class identifier:

```
0, 1, 2 or 3
```

Why Is the Spatial Size the Same?

The output has the same spatial size as the input:

```
128 x 128
```

This is important for semantic segmentation.

The model must produce one prediction for every input pixel.

Therefore, the final output must align with the original image.

This allows direct comparison between:

- predicted mask,
- ground-truth mask.

Why Is This Inspection Important?

Before training, we verify that:

- the model accepts the expected input shape,
- the model produces the expected number of class channels,
- the output spatial resolution matches the mask resolution,
- the architecture is compatible with the loss function.

If the output shape were incorrect, training would fail later when comparing predictions with ground-truth masks.

13 Task 7: Create Training Script

Create:

```
src/segmentation/train_segmentation.py
```

Add imports:

```
from pathlib import Path
import random

import torch
from torch import nn
from torch.utils.data import DataLoader, random_split
from torchvision import transforms

import matplotlib.pyplot as plt
import numpy as np

from src.segmentation.segmentation_dataset import SyntheticSegmentationDataset
from src.segmentation.unet_model import SmallUNet
```

Add configuration:

```
IMAGE_DIR = Path("data/segmentation/images")
MASK_DIR = Path("data/segmentation/masks")

MODEL_PATH = Path("models/small_unet.pt")
REPORT_PATH = Path("reports/segmentation_report.txt")

BATCH_SIZE = 8
EPOCHS = 10
LEARNING_RATE = 0.001
NUM_CLASSES = 4
RANDOM_SEED = 42
```

14 Task 8: Create DataLoaders

Add:

```
def create_dataloaders():

    transform = transforms.Compose([
        transforms.ToTensor()
    ])

    dataset = SyntheticSegmentationDataset(
        image_dir=IMAGE_DIR,
        mask_dir=MASK_DIR,
        transform=transform
    )

    train_size = int(
        0.8 * len(dataset)
    )

    test_size = (
        len(dataset)
        - train_size
    )

    train_dataset, test_dataset = random_split(
        dataset,
        [train_size, test_size],
        generator=torch.Generator().manual_seed(
            RANDOM_SEED
        )
    )

    train_loader = DataLoader(
        train_dataset,
        batch_size=BATCH_SIZE,
        shuffle=True
    )

    test_loader = DataLoader(
        test_dataset,
        batch_size=BATCH_SIZE,
        shuffle=False
    )
```



```

print("=== Segmentation DataLoaders ===")
print(f"Training samples: {len(train_dataset)}")
print(f"Testing samples: {len(test_dataset)}")

images, masks = next(
    iter(train_loader)
)

print(f"Batch image shape: {images.shape}")
print(f"Batch mask shape: {masks.shape}")

return train_loader, test_loader

```

15 Task 9: Select Device

Add:

```

def get_device():

    if torch.cuda.is_available():
        return torch.device("cuda")

    return torch.device("cpu")

```

16 Task 10: Train Segmentation Model

Add:

```

def train_model(
    model,
    train_loader,
    device
):

    loss_function = nn.CrossEntropyLoss()

    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=LEARNING_RATE
    )

    model.train()

    for epoch in range(EPOCHS):

        total_loss = 0.0

        for images, masks in train_loader:

            images = images.to(device)

            masks = masks.to(device)

            optimizer.zero_grad()

            outputs = model(images)

```

```

        loss = loss_function(
            outputs,
            masks
        )

        loss.backward()

        optimizer.step()

        total_loss += loss.item()

    average_loss = (
        total_loss
        / len(train_loader)
    )

    print(
        f"Epoch {epoch+1}/{EPOCHS}, "
        f"Loss: {average_loss:.4f}"
    )

```

17 Task 11: Evaluate Pixel Accuracy

Pixel accuracy measures the percentage of correctly classified pixels.

Add:

```

def evaluate_model(
    model,
    test_loader,
    device
):

    model.eval()

    correct_pixels = 0
    total_pixels = 0

    with torch.no_grad():

        for images, masks in test_loader:

            images = images.to(device)

            masks = masks.to(device)

            outputs = model(images)

            predictions = torch.argmax(
                outputs,
                dim=1
            )

            correct_pixels += (
                predictions == masks
            ).sum().item()

```

```

        total_pixels += masks.numel()

    accuracy = (
        correct_pixels
        / total_pixels
    )

    print("=== Segmentation Evaluation ===")
    print(f"Pixel accuracy: {accuracy:.4f}")

    return accuracy

```

18 Task 12: Save Model and Report

Add:

```

def save_model(model):

    MODEL_PATH.parent.mkdir(
        parents=True,
        exist_ok=True
    )

    torch.save(
        model.state_dict(),
        MODEL_PATH
    )

    print(f"Saved model: {MODEL_PATH}")

```

Add:

```

def save_report(accuracy):

    REPORT_PATH.parent.mkdir(
        parents=True,
        exist_ok=True
    )

    with open(REPORT_PATH, "w") as f:

        f.write("SEMANTIC SEGMENTATION REPORT\n")
        f.write("=====\n\n")

        f.write("Model: Small U-Net\n")
        f.write("Dataset: synthetic EO segmentation dataset\n")
        f.write(f"Classes: {NUM_CLASSES}\n")
        f.write(f"Epochs: {EPOCHS}\n")
        f.write(f"Learning rate: {LEARNING_RATE}\n")
        f.write(f"Pixel accuracy: {accuracy:.4f}\n\n")

        f.write("Interpretation:\n")
        f.write(
            "The model was trained to assign a land-cover class "
            "to every pixel in the image.\n"
        )

```

```
print(f"Saved report: {REPORT_PATH}")
```

19 Task 13: Main Function

Add:

```
def main():

    random.seed(RANDOM_SEED)
    torch.manual_seed(RANDOM_SEED)

    train_loader, test_loader = (
        create_dataloaders()
    )

    device = get_device()

    print(f"Using device: {device}")

    model = SmallUNet(
        num_classes=NUM_CLASSES
    )

    model = model.to(device)

    train_model(
        model,
        train_loader,
        device
    )

    accuracy = evaluate_model(
        model,
        test_loader,
        device
    )

    save_model(model)

    save_report(accuracy)

if __name__ == "__main__":
    main()
```

Run:

```
python3 -m src.segmentation.train_segmentation
```

Expected output:

```
=== Segmentation DataLoaders ===
Training samples: 160
Testing samples: 40
Batch image shape: torch.Size([8, 3, 128, 128])
Batch mask shape: torch.Size([8, 128, 128])
Using device: cpu
```

```
Epoch 1/10, Loss: 1.3521
Epoch 2/10, Loss: 1.1044
...
=== Segmentation Evaluation ===
Pixel accuracy: 0.9000
Saved model: models/small_unet.pt
Saved report: reports/segmentation_report.txt
```

What Is Being Trained?

At this stage, we are finally training the U-Net segmentation model.

The model receives:

```
Input:
RGB image
```

and attempts to predict:

```
Output:
segmentation mask
```

The training data consists of the image-mask pairs generated earlier.

For every training sample:

```
image
U-Net
predicted mask
compare with ground-truth mask
update model parameters
```

The objective is to gradually improve the model's ability to assign the correct class to every pixel.

Understanding the DataLoader Output

The training script first reports:

```
Training samples: 160
Testing samples: 40
```

The original dataset contains:

```
200 image-mask pairs
```

The dataset is automatically split into:

- 160 training samples (80%),
- 40 testing samples (20%).

The training subset is used to learn model parameters.

The testing subset is used to evaluate how well the model generalizes to previously unseen images.

Understanding Batch Shapes

The DataLoader reports:

```
Batch image shape:
torch.Size([8, 3, 128, 128])

Batch mask shape:
torch.Size([8, 128, 128])
```

This means that one training batch contains:

- 8 images,
- 8 corresponding masks.

The image tensor:

```
[8, 3, 128, 128]
```

represents:

- batch size = 8,
- 3 RGB channels,
- image size = 128128.

The mask tensor:

```
[8, 128, 128]
```

contains one class label for every pixel.

What Does the Loss Mean?

During training, the model initially makes mostly random predictions.

The loss function measures how different the predicted mask is from the correct mask.

For example:

Ground truth:

```
water water water
water water water
```

Prediction:

```
urban water urban
water urban water
```

Many pixels are incorrect.

Therefore, the loss is large.

After several training iterations, the model gradually improves.

As predictions become closer to the ground truth, the loss decreases.

This is why we observe:

```
Epoch 1: Loss = 1.35
Epoch 2: Loss = 1.10
Epoch 3: Loss = 0.90
...
```

A decreasing loss usually indicates that the model is learning useful patterns from the data.

What Is Pixel Accuracy?

After training, the model is evaluated on the testing dataset.

The reported metric is:

```
Pixel accuracy
```

Pixel accuracy measures:

How many pixels were assigned the correct class?

For example:

```
Total pixels:
10000

Correctly classified:
9000
```

Then:

```
Pixel accuracy = 0.90
```

which means:

```
90%
```

of all pixels were classified correctly.

What Was Produced?

After training, two important outputs are generated.

Trained Model

```
models/small_unet.pt
```

This file contains the learned neural network parameters.

It can later be loaded and used to predict segmentation masks for new images.

Training Report

```
reports/segmentation_report.txt
```

This report summarizes:

- dataset configuration,
- training parameters,
- segmentation accuracy,

- basic interpretation of results.

20 Task 14: Create Prediction and Visualization Script

Create:

```
src/segmentation/predict_segmentation.py
```

Add imports:

```
from pathlib import Path

import torch
from torchvision import transforms

from PIL import Image
import matplotlib.pyplot as plt
import numpy as np

from src.segmentation.unet_model import SmallUNet
```

Add configuration:

```
MODEL_PATH = Path("models/small_unet.pt")
IMAGE_PATH = Path(
    "data/segmentation/images/scene_0000.png"
)
MASK_PATH = Path(
    "data/segmentation/masks/scene_0000.png"
)

OUTPUT_PATH = Path(
    "reports/segmentation_examples/prediction.png"
)

NUM_CLASSES = 4
```

Add color palette:

```
PALETTE = {
    0: (80, 80, 80),
    1: (40, 140, 40),
    2: (40, 80, 180),
    3: (180, 180, 180)
}
```

21 Task 15: Convert Mask to RGB

Add:

```
def mask_to_rgb(mask):

    height, width = mask.shape

    rgb = np.zeros(
        (height, width, 3),
        dtype=np.uint8
```



```

)

for class_id, color in PALETTE.items():

    rgb[mask == class_id] = color

return rgb

```

22 Task 16: Load Model

Add:

```

def load_model():

    if not MODEL_PATH.exists():
        print(f"Error: model not found: {MODEL_PATH}")
        print("Run train_segmentation.py first.")
        raise SystemExit(1)

    model = SmallUNet(
        num_classes=NUM_CLASSES
    )

    state_dict = torch.load(
        MODEL_PATH,
        map_location="cpu"
    )

    model.load_state_dict(
        state_dict
    )

    model.eval()

    return model

```

23 Task 17: Predict Mask

Add:

```

def predict_mask(
    model,
    image
):

    transform = transforms.Compose([
        transforms.ToTensor()
    ])

    tensor = transform(
        image
    ).unsqueeze(0)

    with torch.no_grad():

        output = model(tensor)

```

```

        prediction = torch.argmax(
            output,
            dim=1
        )

    return prediction[0].numpy()

```

24 Task 18: Visualize Prediction

Add:

```

def visualize(
    image,
    ground_truth,
    prediction
):

    gt_rgb = mask_to_rgb(
        ground_truth
    )

    pred_rgb = mask_to_rgb(
        prediction
    )

    plt.figure(
        figsize=(12,4)
    )

    plt.subplot(1,3,1)
    plt.imshow(image)
    plt.title("Input image")
    plt.axis("off")

    plt.subplot(1,3,2)
    plt.imshow(gt_rgb)
    plt.title("Ground truth mask")
    plt.axis("off")

    plt.subplot(1,3,3)
    plt.imshow(pred_rgb)
    plt.title("Predicted mask")
    plt.axis("off")

    plt.tight_layout()

    OUTPUT_PATH.parent.mkdir(
        parents=True,
        exist_ok=True
    )

    plt.savefig(
        OUTPUT_PATH
    )

    plt.show()

```

```
print(f"Saved prediction: {OUTPUT_PATH}")
```

25 Task 19: Main Function

Add:

```
def main():

    model = load_model()

    image = Image.open(
        IMAGE_PATH
    ).convert("RGB")

    ground_truth = np.array(
        Image.open(MASK_PATH),
        dtype=np.int64
    )

    prediction = predict_mask(
        model,
        image
    )

    visualize(
        image,
        ground_truth,
        prediction
    )

if __name__ == "__main__":
    main()
```

Run:

```
python3 -m src.segmentation.predict_segmentation
```

Expected output:

```
Saved prediction: reports/segmentation_examples/prediction.png
```

26 Task 20: Analyze Segmentation Results

What Are We Looking At?

At this stage, the complete segmentation pipeline has been executed.

The model was trained using image-mask pairs generated earlier in the laboratory.

We now evaluate whether the trained U-Net can successfully reconstruct segmentation masks from previously unseen images.

The generated visualization allows us to compare:

- the original input image,
- the correct segmentation mask,

- the model prediction.

This comparison helps determine whether the model learned meaningful semantic representations rather than simply memorizing the training data. Open:

```
reports/segmentation_examples/prediction.png
```

Compare:

- input image,
- ground truth mask,
- predicted mask.

Answer:

- Which classes were segmented correctly?
- Which classes were most difficult?
- Were object boundaries accurate?
- Did small urban blocks disappear?
- Did the model confuse vegetation and background?
- Did the model confuse water and background?

How to Interpret the Result

The purpose of this experiment is not to achieve state-of-the-art segmentation performance. Instead, it demonstrates the complete workflow of a semantic segmentation system:

```
image  
  
trained model  
  
pixel-level prediction  
  
segmentation mask
```

The generated result should therefore be interpreted as a proof that the pipeline is functioning correctly.

The model successfully identifies large and visually distinctive regions such as water and vegetation.

Small urban structures may be less precise and object boundaries may not perfectly match the ground truth.

This illustrates an important property of semantic segmentation:

Large homogeneous regions are usually easier to segment than small, irregular or fragmented structures.

It is also important to remember that this dataset is synthetic.

Classes are represented by relatively simple colors and geometric patterns.

Real Earth Observation imagery is significantly more challenging due to:

- complex textures,
- illumination changes,
- seasonal variability,
- sensor noise,
- class ambiguity,
- imperfect annotations.

Nevertheless, the workflow implemented in this laboratory is conceptually identical to the workflow used in operational Earth Observation systems.

Documenting Your Observations

Record your conclusions in:

```
reports/segmentation_analysis.txt
```

Use the following template:

```
SEGMENTATION ANALYSIS
=====

Pixel accuracy:

Visual observations:

Most accurate class:

Most difficult class:

Boundary errors:

Small object errors:

Engineering conclusions:
```

Your analysis should combine both quantitative and qualitative evaluation.

Do not focus only on the reported pixel accuracy.

Carefully inspect the generated prediction and describe:

- which classes were segmented correctly,
- which classes caused the largest errors,
- whether object boundaries were preserved,
- whether small structures were detected,
- whether the model confused similar regions,
- what limitations of the current approach can be observed.

The goal is to interpret the model behaviour rather than simply report a numerical score.

This type of analysis is commonly performed during validation of real Earth Observation segmentation systems.

27 Independent Tasks

Independent Task 1: Test More Images

Modify:

```
IMAGE_PATH  
MASK_PATH  
OUTPUT_PATH
```

in:

```
predict_segmentation.py
```

Test at least five different images.

Save visualizations as:

```
reports/segmentation_examples/prediction_1.png  
reports/segmentation_examples/prediction_2.png  
reports/segmentation_examples/prediction_3.png  
reports/segmentation_examples/prediction_4.png  
reports/segmentation_examples/prediction_5.png
```

Questions:

- Are predictions stable across different scenes?
- Do errors repeat across images?
- Which structures are segmented most consistently?

Independent Task 2: Increase Dataset Difficulty

Modify:

```
generate_synthetic_dataset.py
```

Make the dataset more difficult by adding at least one of the following:

- more noise,
- smaller urban blocks,
- thinner water regions,
- overlapping objects,
- more irregular vegetation regions.

Regenerate the dataset.

Retrain the model.

Compare results with the original version.

Questions:

- Did pixel accuracy decrease?
- Which class became harder?

- Did small structures become more difficult?
- What does this suggest about real Earth Observation data?

Save:

```
reports/dataset_difficulty_analysis.txt
```

Independent Task 3: Compute Class-Wise Accuracy

Pixel accuracy gives one global number.

However, it may hide class-specific problems.

Implement class-wise pixel accuracy.

For each class:

```
class accuracy =
correct pixels of this class / all pixels of this class
```

Report accuracy for:

- background,
- vegetation,
- water,
- urban.

Questions:

- Which class achieved the best accuracy?
- Which class achieved the worst accuracy?
- Is global pixel accuracy misleading?
- Why are small classes often harder to evaluate?

Save:

```
reports/class_wise_accuracy.txt
```

Independent Task 4: Compare Classification and Segmentation

Write a short comparison between image classification and semantic segmentation.

Answer:

- What does classification output?
- What does segmentation output?
- Which task is more suitable for land-cover mapping?
- Which task is more suitable for quick scene categorization?
- Why is segmentation more computationally demanding?

- How could segmentation support mission operations?

Save:

```
reports/classification_vs_segmentation.txt
```

28 Submission

Submit:

- source code:

```
src/segmentation/generate_synthetic_dataset.py
src/segmentation/segmentation_dataset.py
src/segmentation/unet_model.py
src/segmentation/train_segmentation.py
src/segmentation/predict_segmentation.py
src/segmentation/visualize_mask.py
```

- trained model:

```
models/small_unet.pt
```

- generated visualizations:

```
reports/segmentation_examples/
```

- main report:

```
reports/segmentation_report.txt
reports/segmentation_analysis.txt
```

- independent task reports:

```
reports/dataset_difficulty_analysis.txt
reports/class_wise_accuracy.txt
reports/classification_vs_segmentation.txt
```

The submitted work should include:

- working implementation,
- trained segmentation model,
- generated prediction examples,
- quantitative results,
- visual interpretation,
- engineering conclusions.